Practice 3: Linear Regression Models

First I used the Hold-Out method to split the datasets (80% training ,20% testing). We are just trying to model general behavior based on some input data, so Hold-Out is fast and simple and perfect for this case.

For metrics, I chose Mean Squared Error (MSE) because it harshly penalizes big mistakes, and the R^2 score to easily see what percentage of the data's variance is actually explained by the models. The goal here was to test out linear regression, alongside L1 (Lasso) and L2 (Ridge) regularizations and see where they fail.

Case 1: Periodic Activity Monitoring (Port Buoys)

This first dataset tracks the changes in water levels over time. In the data plot we can clearly see it follows a wave-like pattern.

Linear vs. Polynomial Performance:

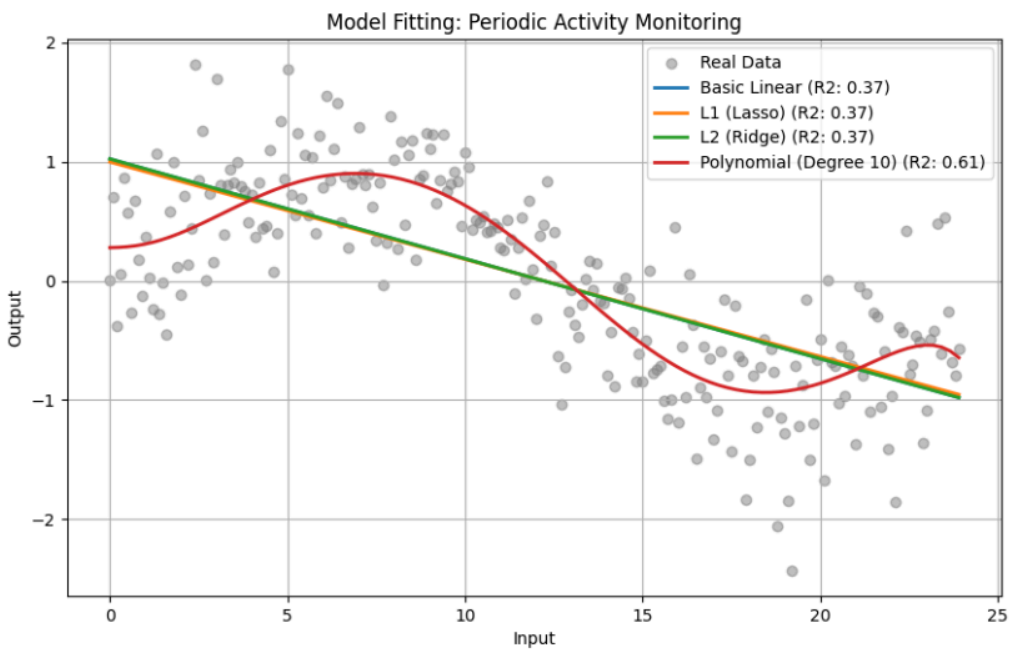
Linear models completely fail in this scenario. It's impossible fit a straight line to a wave. Even when applying L1 or L2 regularization the models suffer from significant underfitting and the R^2 score is extremely low (around 0.37).

To fix this, I used *PolynomialFeatures*. Increasing the degree up to 10 gave the model enough flexibility to curve and follow the peaks and drops of the water level. The R^2 jumped to 0.61, meaning a significant improvement.

Results Table:

Model	MSE	R2 Score
Basic Linear	0.5678	0.3734

L1 (Lasso)	0.5671	0.3742
L2 (Ridge)	0.5678	0.3734
Polynomial (Degree 10)	0.3509	0.6127



Case 2: System Accuracy vs. Training Data

The second dataset shows how a system's success rate improves as you feed it more training data. The shape of this data looks like the typical learning curve; shooting up quickly at the beginning and then flattening out (asymptotic curve).

Linear vs. Polynomial Performance:

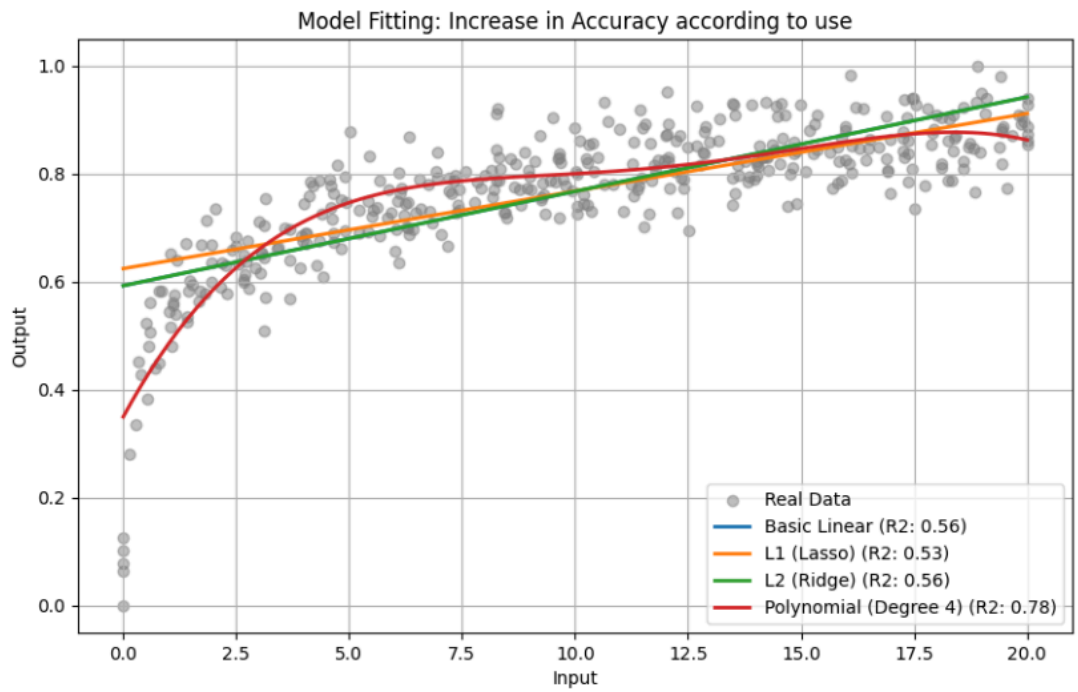
Just like before a simple straight line isn't good enough. Linear models assume the accuracy will keep growing forever at the same rate, which isn't realistic at all. Lasso and Ridge try to flatten

the line a bit to keep the weights low, but they still miss the initial spike and overestimate the end.

By applying a polynomial transformation of degree 4, the model bends perfectly to capture the steep learning phase and a final plateau, boosting the R^2 to 0.78 (from 0.53 in its lowest case).

Results Table:

Model	MSE	R2 Score
Basic Linear	0.0101	0.5587
L1 (Lasso)	0.0107	0.5316
L2 (Ridge)	0.0101	0.5587
Polynomial (Degree 4)	0.0050	0.7825



Final Conclusion

The main takeaway we get from this practice is that standard linear regressors (including the regularized ones) are quite useless taking into account that real-world data is non-linear, periodic, or asymptotic.

By combining the input variables with polynomial transformations, we can avoid these limitations and create models that actually fit the physical events we are trying to predict. On the other hand, picking the right polynomial degree is key; degree 4 was enough for the learning curve, but in the other case we needed degree 10 to capture the complex cycles of the water levels.

References

Console output from the Python script showing the metrics I used for the report.

```
"D:\Users\ptorr\PycharmProjects\Lab 3 Regression\.venv\Scripts\python.exe" "D:\Users\ptorr\PycharmProjects\Lab 3 Regression\code.py"

 Periodic Activity Monitoring
Model Basic Linear: MSE = 0.5678 | R2 = 0.3734
Model L1 (Lasso): MSE = 0.5671 | R2 = 0.3742
Model L2 (Ridge): MSE = 0.5678 | R2 = 0.3734
Model Polynomial (Degree 10): MSE = 0.3509 | R2 = 0.6127

 Increase in Accuracy according to use
Model Basic Linear: MSE = 0.0101 | R2 = 0.5587
Model L1 (Lasso): MSE = 0.0107 | R2 = 0.5316
Model L2 (Ridge): MSE = 0.0101 | R2 = 0.5587
Model Polynomial (Degree 4): MSE = 0.0050 | R2 = 0.7825

Process finished with exit code 0
```